

Physical-Reward Policy Optimization for Accelerator RTL

Mohamed Amine Hallam, Kuo-Kun Tseng, and Wenjie Pei

Abstract—We investigate whether training with a learned physical reward improves the RTL that a policy generates at a fixed sampling budget. During training, an executable oracle screens candidate behavior and gates the physical reward. Independent post-route implementation then evaluates the trained policy, preserving the frequency of each sampled candidate and assigning zero to functional and implementation failures. On a separately frozen 20-design split, two policies initialized from the same supervised fine-tuned (SFT) checkpoint and trained with a canonical-feature random-forest (RF) reward increase failure-penalized timing-derived maximum frequency from 35.4 to 93.0 MHz. The paired gain is 57.6 MHz with a 95% design-bootstrap interval of [37.7, 77.2] MHz. Both RF seeds improve, and matched controls show that the gain cannot be explained by functional yield alone. The RF reward also outperforms the original multilayer-perceptron reward package. A separate setup-closure extension reproduces the positive RF and negative correctness-only effects in two additional seeds per arm, while an earlier Qwen-Coder experiment improves all 22 tested designs. Structural analysis and paired board measurements help explain these gains, which accompany higher resource costs, weak median-filter outcomes, and a small regression on general RTL tasks. The findings support physical reward training within the studied accelerator domain and show why its evaluation must account for failures and use independent physical measurements.

Index Terms: FPGA physical design, hardware-aware optimization, large language models, reinforcement learning, reward-model reliability, RTL generation

I. INTRODUCTION

Language models can generate RTL that passes executable benchmarks [1]–[3], but functional acceptance alone says little about the quality of the implemented circuit. Modules that compute the same outputs can differ substantially in their critical paths, resource mappings, and achievable clock frequencies. A model trained only to pass a testbench has no direct incentive to prefer the faster implementation. Sampling several alternatives and selecting the fastest can address this problem, although it requires repeated electronic design automation (EDA) runs at inference time.

Recent work uses synthesis feedback, reinforcement learning, evolutionary search, and per-design adaptation to improve RTL [4]–[10]. These approaches make it important to distinguish the quality of a selected circuit from the expected quality of a single policy draw. Synthesis-based evaluation also finds a gap between single-attempt and best-of-several results [11]. A learned physical reward offers a way to guide training without

implementing every candidate, but optimization can shift the generated code toward regions where the predictor is less reliable [12]. Its training scores must therefore be assessed against independent physical measurements.

We study whether training with a physical proxy improves expected physical utility at a fixed draw budget in a controlled streaming-accelerator domain. Both functional and implementation failures contribute zero, so the endpoint reflects the full distribution of generated outputs. Matched reward controls test whether physical feedback adds value beyond correctness pressure alone. As shown in Fig. 1, domain SFT initializes the generator, an executable oracle gates the physical reward, and group-relative updates change candidate probabilities. Frozen checkpoints are then evaluated through independent post-route implementation. The contribution lies in this distribution-level evaluation and controlled reward comparison; the policy update and EDA feedback build on established methods.

The primary test is a separately frozen 20-design study spanning interpolation and extrapolation across the retained accelerator generators. Two policy-training seeds sharing one SFT checkpoint and a canonicalization-aware random-forest (RF) reward raise penalized equal-sample F_{max} from 35.4 to 93.0 MHz. Their seed-averaged paired effect is 57.6 MHz with interval [37.7, 77.2] MHz; each training seed is positive, and the interpolation and extrapolation intervals both exclude zero. The optimized policy’s correctness remains close to the supervised fine-tuned (SFT) baseline, and the failure penalty already charges the endpoint for the difference.

The matched controls help explain the gain. The repaired RF reward exceeds the original multilayer-perceptron (MLP) reward by 26.9 MHz under the same tasks and sampling budget. Correctness-only training instead raises validity to 67.4% but lowers penalized F_{max} relative to SFT in its single preregistered seed. The observed physical gain therefore does not arise from higher functional yield. A separate completed setup-closure extension repeats the RF and correctness-only physical directions in additional policy-training seeds, while retaining the shared-SFT limitation. A separate Qwen-Coder [13] experiment improves utility on all 22 tested designs. Conditional resource measurements, paired board checks, and general-RTL evaluation describe the engineering costs and limits of these gains.

Together, these experiments establish a controlled physical improvement on the frozen split and identify its engineering limits. Family and sensitivity analyses show where the gains are weaker, while the original-reward and correctness-only arms distinguish physical improvement from increased functional yield. The comparison supports the RF reward

Mohamed Amine Hallam, Kuo-Kun Tseng, and Wenjie Pei are with the School of Computer Science and Technology, Harbin Institute of Technology (e-mail: 25sf51027@stu.hit.edu.cn; ykkttseng@hit.edu.cn; wenjiecoder@outlook.com).

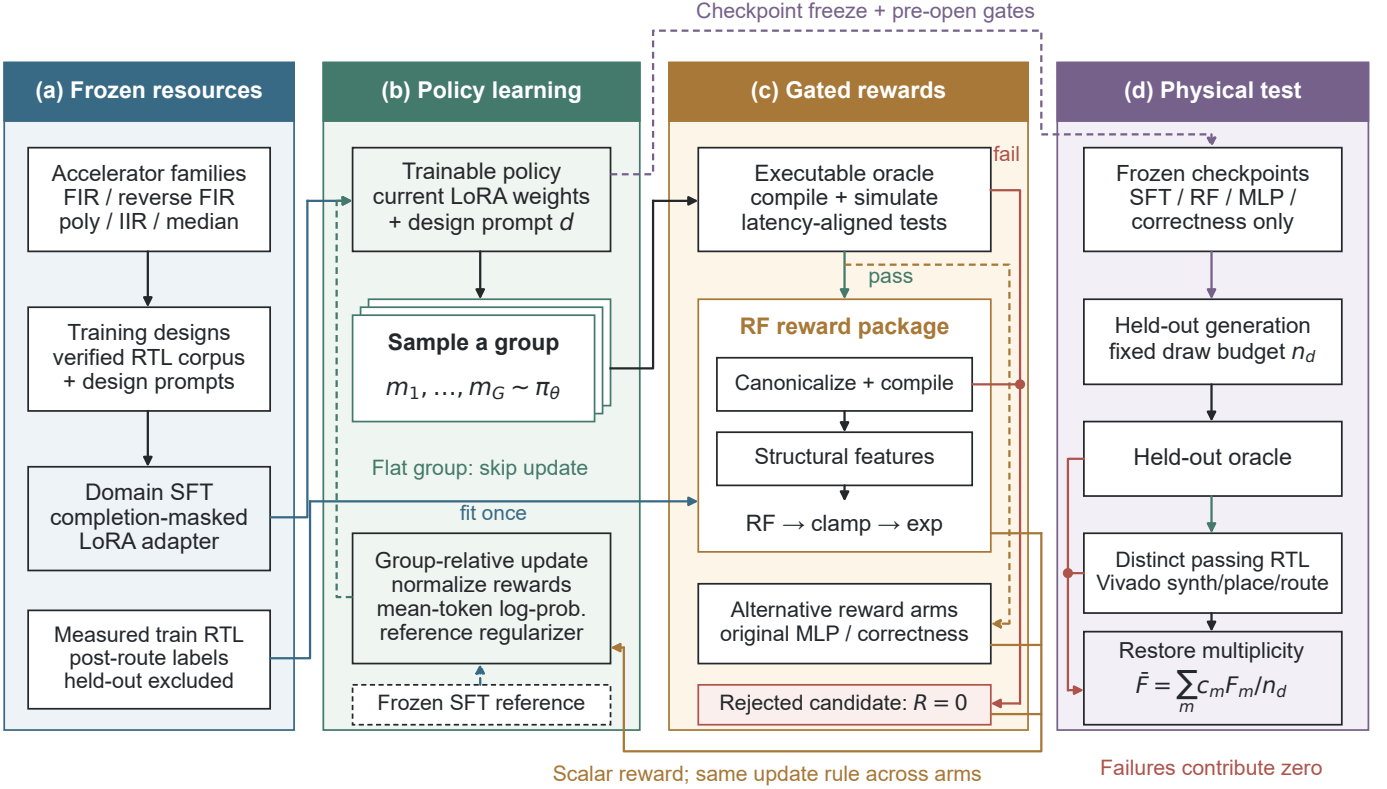


Fig. 1. Implemented training and physical-evaluation workflow. (a) Domain SFT and the physical predictor use training-only resources. (b) Fresh policy samples receive group-relative updates with a frozen SFT reference; flat groups do not advance the update counter. (c) The RF reward uses canonicalized structural features; MLP and correctness-only rewards are separate control runs. Red branches assign zero to rejected candidates. (d) Frozen checkpoints are evaluated using the executable oracle and post-route implementation. Original multiplicity and the full draw denominator are restored; incorrect and implementation-failed draws are not discarded. Dashed paths denote feedback, references, or separately run controls.

package as a whole; it does not isolate predictor architecture from canonical features. Throughout the evaluation, the fixed-draw endpoint preserves sample multiplicity and assigns zero to every incorrect or implementation-failed draw. Post-route measurements determine the reported outcome, with source-level RTL analysis, conditional PPA, setup checks, board measurements, and general-RTL evaluation providing context for its practical value.

The candidate analysis suggests that much of the gain comes from making fast implementations more likely. Supervision already supplies several passing forms, and optimization concentrates probability on a smaller set of outputs. The sealed support diagnostic nevertheless contains exceptions, so the finite SFT sample cannot account for every observed gain. These results establish a change in the sampled distribution without resolving whether optimization produces new circuit structures. They also explain why the benefit depends on the implementations available to the generator and on the reward’s ability to distinguish their physical quality.

II. RELATED WORK

A. Executable RTL generation

VerilogEval and RTLLM established executable benchmarks for RTL generation [1], [2]. The revised VerilogEval adds specification-to-RTL tasks, in-context examples, and failure classification [3]. ChipVerilog extends evaluation to

larger OpenCores-derived designs and cross-module interactions [14]. OpenLLM-RTL expands the RTLLM benchmark and releases verified training data [15]. TuRTL integrates benchmarks to evaluate syntax, functionality, synthesis, and PPA within a common framework [16]. VeriGen, RTLCode, CodeV, and CraftRTL improve generation through specialized pretraining, supervised adaptation, data construction, or targeted repair [17]–[20]. VeriSeek introduces code-structure-guided reinforcement learning, while VeriReason combines supervised learning, testbench feedback, and group-relative policy optimization [21], [22]. VeriPrefer uses testbench outcomes to construct preference pairs for correctness-oriented training [23]. FormalRTL couples executable specifications to formal verification in a multi-agent synthesis flow [24]. VeriRefine makes specification refinement an explicit stage and routes failures back to either interpretation or code generation [25]. These approaches improve or assess functional generation. Our study asks how physical quality changes across a policy’s sampled outputs, retaining both oracle rejections and implementation failures in the evaluation.

B. LLM-driven physical optimization

Physical feedback can guide training, target-specific adaptation, or search. BetterV uses task-specific generative discriminators to guide Verilog generation for synthesis and verification objectives [26]. RTLRewriter rewrites an existing

implementation [6]. SymRTLO combines LLM rewriting with symbolic reasoning and formal/test-based validation for physical optimization [27]. PPA-RTL constructs synthesis-scored preference pairs and applies direct preference optimization (DPO) [4]. ChipSeek uses curriculum-guided dynamic policy optimization with syntax, functional, and synthesis-derived feedback [5]. Alpha-RTL instead adapts the policy for each target at test time through an executable EDA loop [9]. Dr. RTL uses commercial-tool feedback, critical-path analysis, parallel rewriting, and reusable optimization skills [10]. POET and Evolve apply evolutionary search to verified RTL variants [7], [8]. REvolution evolves separate populations for bug fixing and PPA improvement [28]. COEVO jointly searches correctness and PPA using an adaptive correctness gate and Pareto-based selection [29]. RTL-OPT contributes a benchmark of suboptimal/human-optimized RTL pairs and an automated correctness/PPA evaluation flow [30]. Fu et al. assess synthesis quality using area, delay, and tool warnings, and distinguish single-attempt quality from best-of-several performance [11]. Their evaluation reinforces the need to specify how sampled outcomes are aggregated; our endpoint uses post-route FPGA timing and retains each draw’s original multiplicity.

Table I distinguishes selected implementations from policy distributions. ChipSeek reports mean and extreme physical metrics among correct generations; our frozen paired evaluation retains the full draw denominator, including failures. The perfect-selector analysis separately compares repeated SFT sampling with one optimized draw. The internal reward controls do not establish head-to-head superiority over PPA-RTL or ChipSeek.

C. Physical predictors and reward reliability

Pre-synthesis predictors reduce the cost of FPGA design-space exploration [31]–[33]. Optimizing a learned reward changes its input distribution and can magnify prediction errors [12]. In our method, the predictor guides training, while post-route and board measurements provide physical evidence. The historical reward trajectory illustrates why this separation matters, without implying that physical predictors fail in general. Our contribution is a controlled evaluation of physical-reward training at a fixed draw budget, supported by matched reward controls, paired design-level analysis, and candidate diagnostics. The resulting conclusions apply to the studied streaming accelerators and sampled oracle contract; they do not establish formal equivalence or superiority on industrial RTL.

III. METHOD

A. Problem formulation and overview

Our goal is to improve the expected physical utility of generated accelerator RTL at a fixed draw budget. For specification d , policy π_θ samples completion m . Let $g_d(m)$ be the binary executable-oracle verdict and $F(m)$ the post-route timing-derived frequency, defined as zero if implementation fails. The objective over a design distribution $\mathcal{D}$ is

$$\max_{\theta} J(\theta), \quad J(\theta) = \mathbb{E}_{d \sim \mathcal{D}} \mathbb{E}_{m \sim \pi_\theta(\cdot|d)} [g_d(m)F(m)]. \quad (1)$$

Because failures contribute zero, this objective favors policies that assign more probability to fast implementations that pass the oracle. Equation (6) gives the finite-sample estimate used in the evaluation. Implementing every training sample would be too expensive, so a learned surrogate supplies the training reward and measured post-route timing determines the reported result. Fig. 1 places these steps in context. Domain SFT provides both the initial policy and a frozen reference; fresh completions pass through the executable gate and receive a physical or control reward. Group-relative updates then change the policy’s sampling distribution. The final comparison evaluates frozen checkpoints at equal draw budgets using the oracle and Vivado.

Each accelerator family has a deterministic software reference. The oracle compiles generated modules and compares simulated outputs on frozen stimuli. To accommodate different pipeline depths, it searches a bounded latency window and accepts the best aligned match fraction at or above 0.999. The search allows zero through 40 cycles of latency and discards 8 warm-up outputs. A sufficiently long trace can pass this near-unity threshold despite a mismatch. We use “correct” below to mean oracle-passing, not formally equivalent. The same oracle filters supervised examples, gates policy rewards, and evaluates held-out samples, with held-out stimulus streams reserved for evaluation. Let $C_d(m)$ denote the binary oracle verdict. Every reward arm obeys

$$g_d(m) = \begin{cases} 1, & C_d(m) \text{ passes,} \\ 0, & C_d(m) \text{ fails.} \end{cases} \quad (2)$$

The gate rejects behavior changes detected by its sampled tests. It neither proves equivalence on all inputs nor establishes that a proxy ranks admissible modules properly; physical implementation addresses the latter question.

B. Physical reward and matched controls

The physical reward ranks oracle-passing completions. It first applies a frozen lexical canonicalizer κ , then predicts log-frequency from canonical structural features ϕ :

$$R_d^{\text{RF}}(m) = g_d(m) \exp \left[\text{clip} \left(\widehat{f}_{\text{RF}}(\phi(\kappa(m))), a, b \right) \right]. \quad (3)$$

Here $\widehat{f}_{\text{RF}}$ predicts log-frequency, a, b are the frozen log-frequency clamp bounds, and exponentiation returns the reward in MHz. The RF contains 300 trees and uses 15 features: multiplier count, maximum per-statement multiplier count, positive-edge count, nonblocking and blocking assignment counts, multiply-accumulate structure, register and wire counts, ternary and comparison counts, statement count, a structural pipeline ratio, and loop count, maximum bound, and summed bounds. Descriptive impurity importance ranks multiplier count, multiply-accumulate structure, pipeline ratio, register count, and nonblocking assignments highest; this ranking describes the fitted predictor and is not a causal circuit attribution. It is fitted to 229 measured RTL rows that exclude every held-out and sealed design. Layout-dependent line counts and name-dependent features are absent. The canonicalizer is lexical and layout-invariant under its frozen

TABLE I

PROTOCOL-LEVEL POSITIONING AGAINST THE CLOSEST PHYSICAL RTL LEARNING AND OPTIMIZATION SETTINGS. “SELECTED IMPLEMENTATION” DENOTES THE OUTPUT OF A TARGET-SPECIFIC SEARCH; “POLICY DISTRIBUTION” RETAINS EVERY SAMPLED OUTCOME AND ITS MULTIPLICITY.

Approach	Optimization locus	Feedback/reporting authority	Primary object
PPA-RTL [4]	Fixed-preference DPO	Synthesis-scored preference pairs	Objective-conditioned RTL generation
ChipSeek [5]	Curriculum-guided policy optimization	Syntax, simulation, and synthesis feedback	Correctness and conditional physical quality
Alpha-RTL [9]	Per-design test-time learning	Executable verification and EDA feedback	Selected implementation after adaptation
Dr. RTL [10]	Agentic per-design rewriting	Commercial EDA and critical-path feedback	Selected optimized implementation
RTL-OPT [30]	Evaluation benchmark	Automated equivalence and PPA evaluation	Suboptimal-to-reference optimization
This work	Proxy-guided on-policy training	Executable training gate; post-route EDA for reporting	Equal-sample policy distribution on a frozen split

mutation suite; it is not a canonical hardware representation. Canonicalization or canonical-compilation failure also sets the reward to zero and is logged.

An earlier reward failure motivated the choice of canonical features and the RF predictor, as documented in the Supplementary Material. We fixed these features and hyperparameters before opening the sealed efficacy split, which allowed the repair hypothesis to be tested prospectively on that split. The hypothesis itself was informed by the earlier outcomes. The RF provides reward variation in 77.6% of eligible training groups. The supplement also records the reward-path validation and the changes to its scope before sealed evaluation.

Two controls assess the role of the reward. The original-MLP arm restores the previous textual/structural predictor while keeping the SFT initialization, prompts, oracle, optimizer, SFT reference, update target, and evaluation fixed. The correctness-only arm uses $R_d^C(m) = g_d(m)$ and therefore tests whether correctness pressure suffices in the realized training run. Because canonicalization, features, and predictor class change jointly, RF versus MLP is not a component-wise ablation. The RF arm has 2 independent training seeds, the MLP arm has 2, and the correctness-only arm has 1 preregistered full-budget seed.

C. Supervised initialization and policy training

We adapt RTLCoder [18] on an oracle-verified accelerator corpus using completion-masked LoRA [34]. Held-out designs are excluded from the corpus, reward-training rows, and policy prompts. The supervised adapter supplies both the policy initialization and the frozen reference, allowing optimization to remain anchored to the domain competence established by supervision. Using the raw base model as the reference would instead regularize toward a model that had not received this domain adaptation.

Each training group contains 8 completions sampled at temperature 1.0, with at most 1536 completion tokens. If R_i is the gated reward of completion m_i , its within-group advantage is

$$A_i = \frac{R_i - \bar{R}}{s_R + \epsilon}. \quad (4)$$

We sample each attempted group from the current policy and use its rewards directly in the update. Groups without reward

variation provide no ranking information and do not advance the optimizer-update counter. For an informative group, the implemented objective uses the mean completion-token log probability $\ell_\theta(m \mid d) = |m|^{-1} \sum_i \log \pi_\theta(m_i \mid d, m_{<i})$:

$$\mathcal{L}_i = -A_i \ell_\theta(m_i \mid d) + \beta (\exp(\Delta_i) - \Delta_i - 1), \quad (5)$$

where $\Delta_i = \ell_{\text{SFT}}(m_i \mid d) - \ell_\theta(m_i \mid d)$ and the reference is detached. Losses are averaged within the group. This length-normalized update is inspired by group-relative policy optimization [35]; it is not the standard clipped GRPO objective. Exponentiating a difference of mean token log probabilities does not yield the usual sequence-probability ratio, so the penalty is a reference regularizer rather than an unbiased KL estimate. Every arm uses AdamW with learning rate 1e-5, reference-penalty coefficient 0.1, and a target of 276 non-flat updates. Checkpoints follow the training update counter rather than attempted groups; the frozen midpoint is update 138.

D. Physical evaluation and distribution analysis

Evaluation implements each distinct oracle-passing candidate once and restores its original sample multiplicity. For policy p and design d , let $\mathcal{M}_{p,d}$ be the distinct correct candidates, c_m the sampled multiplicity, n_d the complete draw budget, and $F(m)$ the post-route timing-derived frequency, defined as zero when implementation fails. The primary per-design endpoint is

$$\bar{F}_{p,d} = \frac{\sum_{m \in \mathcal{M}_{p,d}} c_m F(m)}{n_d}. \quad (6)$$

Incorrect samples are absent from $\mathcal{M}_{p,d}$ but remain in n_d and therefore contribute zero. Each arm receives 48 draws per design; a two-seed arm uses 24 draws from each seed. The headline statistic is the unweighted mean of $\bar{F}_{p,d}$ across designs, so the design rather than the number of distinct candidates is the aggregation unit. Each candidate is implemented once under the same frozen clock request; $F(m)$ is therefore derived from that request and worst negative slack rather than from candidate-specific repeated timing closure. This symmetry supports the paired comparison but bounds absolute-frequency claims to the frozen flow, device, and request.

The endpoint decomposes as

$$\bar{F}_{p,d} = q_{p,d} \mu_{p,d}, \quad (7)$$

where $q_{p,d}$ is empirical correctness and $\mu_{p,d}$ is multiplicity-weighted frequency conditional on an oracle-passing draw, retaining zero for implementation failure. We set $\mu_{p,d} = 0$ when no draw passes. A policy that rarely emits very fast correct RTL can have high $\mu_{p,d}$ and low $\bar{F}_{p,d}$; reporting only the former would change the estimand.

For paired uncertainty, optimized-minus-SFT differences are computed per design and resampled within frozen family-by-regime strata. The primary plan uses 100,000 resamples and a 95% percentile interval. The direct RF-minus-MLP analysis reuses the identical resample plans but is labeled post-primary. Candidate draws are not treated as independent experimental units, and these design-level intervals do not estimate variation from SFT training, reward-data construction, or model-family choice.

Post-primary diagnostics examine how probability is distributed over passing candidate strings and compare one RF draw with a perfect selector over repeated SFT draws. This selector uses the measured finite SFT support, retains failures as zero-valued outcomes, and assumes that physical measurements are available before selection. It therefore describes an inference workload on the observed support rather than a complete training-and-deployment cost comparison. The Supplementary Material gives the exact calculation, implementation counts, and artifact checks; training cost is excluded from these workloads.

IV. EXPERIMENTAL PROTOCOL

A. Designs, training arms, and sampling

The benchmark covers streaming finite-impulse-response (FIR), reverse-FIR, polynomial, recursive-filter, and median-filter accelerators under a uniform clocked interface. The families cover feed-forward arithmetic pipelines, recurrences, and comparator networks. Each specification and its executable reference are generated programmatically, and several implementation styles provide physically distinct but behaviorally equivalent training examples. This construction lets us compare several accelerator structures within a controlled domain, while keeping the scope distinct from arbitrary industrial RTL.

The sealed evaluation contains 20 designs, balanced between 10 interpolation and 10 extrapolation tasks. Family quotas are unequal because the eligible median interpolation pool was empty; the median family is represented only under extrapolation, so median-family and regime effects cannot be separated. The family breakdown in Sec. V reports the exact composition. Interpolation changes parameters within training ranges; extrapolation uses larger or newly parameterized instances outside those ranges. Every sealed design is excluded from the SFT corpus, reward rows, policy prompts, and prior analysis. The split remained unopened until all declared checkpoints and pre-open validation artifacts were frozen.

The primary comparison starts from RTLCoder and its completion-masked SFT adapter, and evaluates two independently trained repaired-RF policy adapters. The MLP arm

trains two matched policies from the same SFT adapter using the original reward. The correctness-only arm uses one preregistered full-budget seed. Arms use the same non-flat training-update target; flat groups provide no policy-ranking information and do not advance the counter. A crashed run is discarded and restarted from the frozen SFT state; partial histories are never merged. Equal update targets do not imply equal training compute: flat groups still require generation and oracle execution, particularly for correctness-only rewards. The primary comparison matches deployment draws, not end-to-end training time or energy.

At the sealed endpoint, SFT and every trained arm receive 48 draws per design at temperature 1.0. Two-seed arms contribute 24 draws per seed. The midpoint diagnostic uses 8 draws per RF seed and design. Generation seeds are distinct from training seeds and from the 2 oracle streams, each of which contains 1,024 test vectors. Accepted candidates are deduplicated only for implementation; multiplicity is preserved for every policy statistic.

B. Post-route evaluation and statistical analysis

We use the frozen physical flow 2023.1 (laptop, xc7z020clg400-1). Every distinct correct candidate is synthesized, placed, and routed with the same 5.0-ns requested clock. Achievable frequency is derived from the request and reported worst negative slack. We therefore use “post-route timing-derived F_{max} ” rather than “timing closure”: the flow does not repeatedly rerun implementation at candidate-specific clock periods. The historical script introduces the clock constraint after synthesis; this flow is held fixed rather than retroactively corrected in the sealed result.

The primary estimand is Eq. (6). Correctness and conditional frequency are diagnostics, not replacement endpoints. The primary RF-minus-SFT confidence plan is frozen and stratified by family and regime, with percentile bounds. RF-minus-MLP intervals reuse its exact resample plans as a bounded post-primary comparison. The family cells are small and unequal, so the manuscript adds an unstratified paired-bootstrap and leave-one-family-out sensitivity but does not replace the frozen primary plan. No multiplicity adjustment is applied: the RF-versus-SFT contrast is the sole prespecified primary inference, while control, family, PPA, and mechanism analyses are secondary or descriptive. Resource and power analyses condition on designs with compiled correct support in SFT, MLP, and RF; assigning zero area or power to a failed sample would incorrectly make failure look efficient. Power is Vivado’s vectorless estimate, not measured board power.

C. Physical validation and additional seeds

Three supporting studies probe physical sensitivity. A PYNQ-Z2 experiment uses paired SFT/RF candidates selected by multiplicity and an outcome-independent hash. All DUTs and an echo canary share one image and clock grid. Eligibility and sweep coverage were amended before board measurement, so the subset is descriptive. The Supplementary Material records the harness and amendments. A separate pilot brackets setup-passing clock periods for 10 selected candidates under

Vivado 2026.1, with constraints applied before synthesis. An exhaustive cross-version diagnostic retains the original RTL, device, clock request, and constraint order, and checks setup-path coverage separately.

The additional-seed study evaluates RF seeds three and four and correctness-only seeds two and three on all 20 frozen designs. SFT is remeasured under the same bounded setup-period flow, retaining each checkpoint’s original draws and all physical failure zeros. The physical packages were frozen before measurement; the statistical specification followed inspection of point estimates. Its exploratory paired design-bootstrap intervals use 100000 shared family-by-regime resamples. They condition on the sampled checkpoints and are reported separately from the original timing-derived endpoint. None of these studies establishes full hold and pulse-width timing sign-off.

D. Transfer and supporting evidence

To assess transfer beyond the main backbone, we evaluate the complete SFT-to-policy recipe on Qwen-Coder [13] using the earlier 22-design, 3-family split. This second-backbone experiment uses an earlier reward/training configuration, so it tests transfer of the recipe rather than replication of the sealed RF package. A matched VerilogEval extension tests the sealed SFT and RF checkpoints on 156 official problems using 20 draws per checkpoint and problem. The Supplementary Material preserves the earlier five-family evaluations, predictor diagnostics, API and HLS context, and detailed protocol history. Those studies have distinct sampling or selection rules and are not pooled with the sealed estimate. All reported numerical values are generated from retained artifacts; complete denominators, failures, source hashes, and claim provenance are checked during manuscript regeneration.

V. RESULTS

A. Sealed efficacy, controls, and additional seeds

The repaired-RF policy improves the matched physical endpoint across the 20 held-out designs (Table II). Penalized equal-sample F_{max} rises from 35.4 to 93.0 MHz. The paired gain is 57.6 MHz with interval [37.7, 77.2] MHz. The gain appears in both regimes: interpolation improves by 77.9 MHz with interval [48.4, 107.2], while extrapolation improves by 37.4 MHz with interval [13.9, 60.9] MHz. Fig. 2 shows every design together with the frozen paired analysis. Zero-valued designs remain in the figure and in the aggregate, so the reported improvement accounts for failures as well as successful hardware.

Both independently trained RF policies improve, with overall effects of 49.5 and 65.8 MHz. Their positive directions also hold separately under interpolation and extrapolation (with seed-level tables in the Supplementary Material). Correctness changes from 56.2% for SFT to 53.9% for RF. The primary endpoint accounts for this change by assigning zero to every oracle-rejected draw. At the family level, effects are positive for FIR, reverse FIR, polynomial, and recursive filters, and negative for median filters, which appear only in extrapolation. All leave-one-family-out mean effects remain positive, from

46.0 to 69.7 MHz; detailed cells and bootstrap sensitivities appear in the Supplementary Material.

The endpoint decomposition helps explain the gain. RF increases pooled conditional frequency from 62.89 to 172.71 MHz despite its lower oracle pass rate. Correctness-only reaches 36.62 MHz conditional frequency. The improvement therefore comes from the physical quality of passing draws. Fig. 3 makes this distinction explicit: correctness-only training reaches 67.4% correctness, above both SFT and RF, but attains only 24.7 MHz penalized F_{max} . Its paired SFT-relative effect is -10.7 MHz with interval [-14.3, -7.2] MHz. A higher oracle pass rate alone therefore does not explain the RF gain. The additional seeds reported below test whether this contrast persists across training runs.

The original MLP reward does improve over SFT, reaching 66.1 MHz with a paired gain of 30.8 MHz. The repaired RF nonetheless exceeds it directly by 26.9 MHz with interval [16.2, 37.5] MHz. The RF-minus-MLP intervals remain above zero under interpolation and extrapolation as well. Because the arms share their SFT initialization, tasks, update budget, oracle, and evaluation, this comparison favors the RF reward package for these checkpoints.

The post-primary extension in Table III compares the additional seeds with SFT under the same setup-closure flow, retaining the 4 physical failures as zero-valued outcomes. RF reaches 99.14 and 92.98 MHz against SFT’s 36.68 MHz. Correctness-only reaches 24.43 and 25.12 MHz. Both RF paired intervals are above zero and both correctness-only intervals are below zero (Fig. 4), repeating the directions of the original comparison. These runs share the original SFT initialization, and their physical flow differs from the sealed primary flow. Their endpoints and exploratory intervals are therefore reported separately from the primary result.

B. Physical costs and validation

Higher frequency comes with a resource trade-off. On the 15-design common support, RF uses 754.9 rather than 689.2 LUTs and 236.2 rather than 161.4 flip-flops. Conversely, DSP use falls from 5.00 to 2.73 blocks, while BRAM remains unchanged. Vectorless power rises from 0.160 to 0.162 W. The paired intervals in Table IV show that the LUT, register, DSP, and vectorless-power directions are stable on common support. These conditional quantities answer what a compiled correct implementation costs; they do not replace the failure-penalized policy endpoint.

The RTL pair in Table V illustrates this trade-off. For fir13_v6_8b, the dominant SFT candidate stores delayed samples and products before a wide parallel sum. The dominant candidate from RF seed one instead emits a registered transposed multiply-accumulate chain. Vivado maps the latter to substantially more LUTs and registers, no DSP blocks, and a much shorter timing path. Because this case was selected post-primary for interpretability, it illustrates a physically meaningful RTL transformation but does not establish that every family or every gain follows the same transformation.

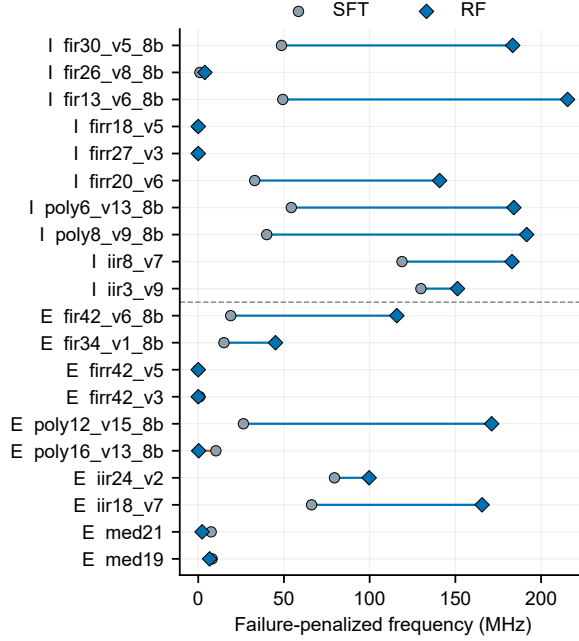
Physical sensitivity checks support relative agreement with limits on absolute frequency. The selected setup-closure pilot

TABLE II

HEADLINE SEALED STUDY 2 COMPARISON OVER THE SAME 20 DESIGNS. PENALIZED F_{max} IS AVERAGED EQUALLY OVER DESIGNS AND SCORES INCORRECT OR IMPLEMENTATION- FAILED SAMPLES AS ZERO. CONFIDENCE INTERVALS ARE PAIRED, STRATIFIED 95% BOOTSTRAP INTERVALS VERSUS THE SFT BASELINE.

Arm	Training seeds	Penalized Fmax [MHz]	Correct [%]	Δ vs. SFT [95% CI]
SFT	—	35.4	56.2	—
Correctness only	1	24.7	67.4	-10.7 [-14.3, -7.2]
Original MLP reward	2	66.1	49.9	30.8 [14.1, 46.7]
Repaired RF reward	2	93.0	53.9	57.6 [37.7, 77.2]

(a) Per-design physical endpoint



I: interpolation E: extrapolation

(b) Paired RF – SFT effects

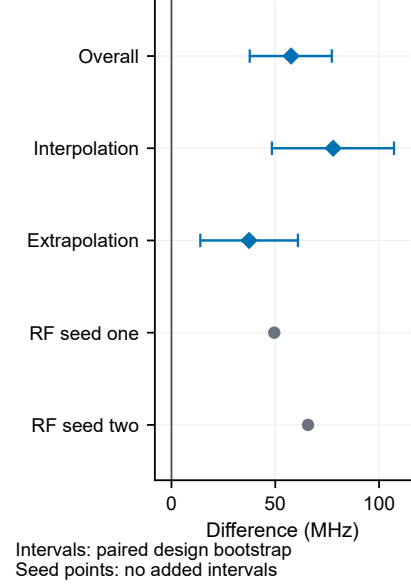


Fig. 2. Sealed primary efficacy. Panel (a) retains every design and joins its SFT and repaired-RF penalized equal-sample post-route F_{max} . Panel (b) reports the frozen paired effects and the separately displayed training-seed effects. The overall gain is 57.6 MHz [37.7, 77.2] MHz.

TABLE III

COMPLETED POST-PRIMARY SETUP-CLOSURE EXTENSION. UTILITY AND PAIRED DIFFERENCES ARE IN MHz; INTERVALS ARE EXPLORATORY 95% DESIGN-BOOTSTRAP INTERVALS CONDITIONAL ON CHECKPOINTS. FAIL: DISTINCT PHYSICAL FAILURES.

Policy	Utility	Difference vs. SFT [CI]	Fail
SFT	36.68	Reference	1
RF seed three	99.14	62.46 [35.56, 90.66]	1
RF seed four	92.98	56.30 [38.28, 73.84]	1
Correctness seed two	24.43	-12.25 [-15.98, -8.66]	0
Correctness seed three	25.12	-11.56 [-15.42, -7.86]	1

gives Spearman correlation 0.963, median symmetric error 3.50%, and no reversal among 3 non-tied pairs. Its worst candidate error is 45.56%, on reverse FIR. Separately, the cross-version coverage diagnostic confirms a setup path and clear constraint counts for 197 of 202 candidates; 1 lacks a path and 4 remain inconclusive. On common valid candidates, median symmetric errors are 0.358% for SFT, 0.697% for RF seed one, and 0.612% for RF seed two. The Supplementary Material gives all cases and preserves the distinction between

diagnostic agreement and a revised primary endpoint.

The paired board measurements provide a further physical check (Table VI and Fig. 5). Across 3 complete sweeps, the maximum within-entry boundary spread is 0.0 MHz. The echo canary reaches 200.0 MHz and remains at least 57.1 MHz above every DUT boundary. The descriptive mean changes from 58.8 to 90.7 MHz, a +31.9-MHz paired difference driven by two large feed-forward gains. Polynomial and IIR tie, while the median pair changes by -9.7 MHz. The board results thus support the overall physical direction but show substantial variation across families. As a descriptive subset measured under the amended protocol in the Supplementary Material, they do not independently confirm the population estimate.

C. Structural changes and sampling efficiency

The sampled candidates suggest that optimization often increases the probability of implementations already accessible to SFT. Among 16 designs with passing sampled strings under both policies, SFT's measured support contains a candidate at least as fast as the RF conditional mean on 14. Mean inverse-Simpson effective correct-string support contracts from 3.78

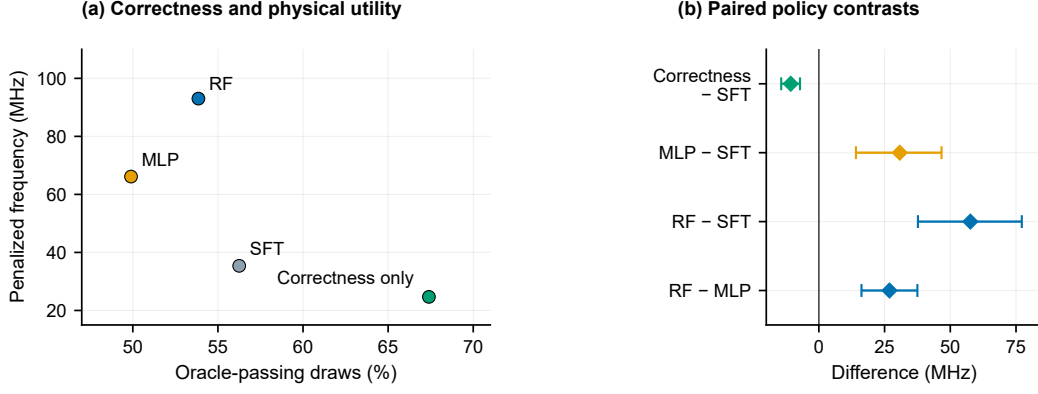


Fig. 3. Matched controls. Panel (a) plots correctness against the failure-penalized endpoint; the correctness-only arm is the key counterexample. Panel (b) gives paired contrasts. Repaired RF exceeds the original MLP reward by 26.9 MHz [16.2, 37.5] MHz.

TABLE IV

CONDITIONAL PHYSICAL-COST DIAGNOSTICS ON THE 15-DESIGN THREE-ARM COMMON SUPPORT. VALUES ARE MULTIPLICITY-WEIGHTED WITHIN DESIGN AND EQUALLY WEIGHTED ACROSS DESIGNS. POWER IS VIVADO'S VECTORLESS ESTIMATE, NOT BOARD POWER.

Metric	SFT	MLP	RF	RF-SFT difference [CI]
LUTs	689.2	766.7	754.9	65.7 [47.2, 84.1]
Flip-flops	161.4	222.3	236.2	74.8 [66.1, 83.4]
DSP blocks	5.00	4.06	2.73	-2.26 [-2.82, -1.70]
BRAM blocks	0.00	0.00	0.00	0.00 [0.00, 0.00]
Vectorless power [W]	0.160	0.168	0.162	0.003 [0.002, 0.003]

TABLE V

ILLUSTRATIVE SOURCE-LEVEL CONTRAST FOR fir13_v6_8b. THE ENTRIES ARE THE DOMINANT SFT CANDIDATE AND DOMINANT CANDIDATE FROM RF SEED ONE FOR THAT DESIGN. SELECTION OCCURRED AFTER MANUSCRIPT ASSESSMENT; THIS EXAMPLE EXPLAINS A POSSIBLE CAD MECHANISM BUT IS NOT A POPULATION OR CAUSAL ESTIMATE.

Arm	Observed RTL organization	Sample mass [%]	Fmax [MHz]	LUT	FF	DSP
SFT	Delayed samples and product registers followed by a parallel final sum	45.8	47.4	1	88	13
RF seed one	Registered transposed multiply-accumulate chain	87.5	220.1	251	221	0

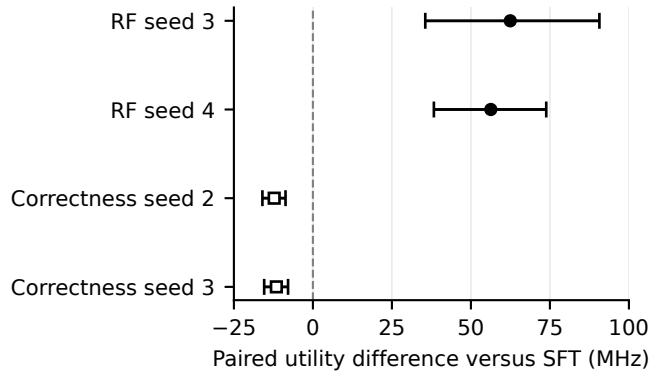


Fig. 4. Separate post-primary seed extension. Points are paired per-design utility differences; bars are exploratory 95% family-by-regime design-bootstrap intervals. They condition on the evaluated checkpoints and sampled draws.

to 2.67. Together, these observations are consistent with a shift toward faster implementations on most designs with common support. The samples are finite, however, and some

TABLE VI

DESCRIPTIVE PYNQ-Z2 CLOCK-SWEEP CONFIRMATION ON 5 PAIRED DESIGNS. VALUES ARE MEDIANS OVER 3 REPEATED SWEEPS. THE SUBSET REQUIRED A DISCLOSED CORRECTNESS-FEASIBILITY AMENDMENT AND IS NOT INDEPENDENT CONFIRMATORY EVIDENCE.

Family	Design	SFT	RF	RF-SFT [MHz]
FIR	fir34_v1_8b	40.0	125.0	+85.0
FIRR	firr20_v6	58.8	142.9	+84.0
POLY	poly16_v13_8b	30.3	30.3	+0.0
IIR	iir18_v7	125.0	125.0	+0.0
MED	med19	40.0	30.3	-9.7

sealed designs fall outside this pattern, leaving circuit novelty unresolved.

On the sealed distribution, one RF draw lies between the empirical SFT perfect-selector expectations for 17 draws (92.47 MHz) and 18 draws (93.66 MHz). At the upper integer crossing, the SFT workload is 18 generations and 36 oracle simulation streams per design, with 10.125 expected correct-draw implementation attempts without caching or 3.078 distinct implementations with content caching. These

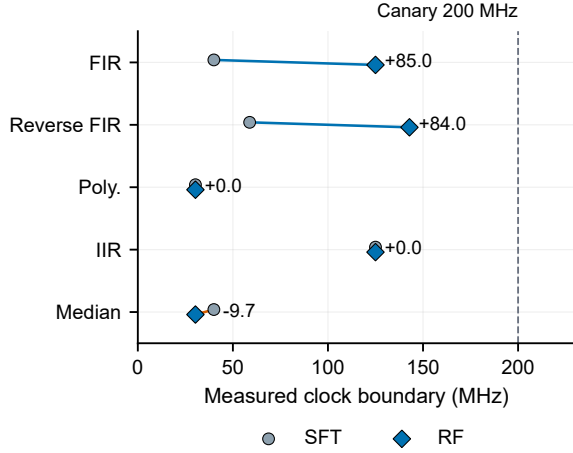


Fig. 5. Descriptive PYNQ-Z2 clock boundaries under the amended protocol. All pairs are retained: 2 RF wins, 2 ties, and 1 loss. Annotations are signed RF-minus-SFT differences in MHz; the dashed line is the echo-canary boundary. No population interval is inferred from this subset.

TABLE VII

SECOND-BACKBONE REPLICATION ON THE EARLIER THREE-FAMILY SPLIT. VALUES ARE PENALIZED EQUAL-SAMPLE POST-ROUTE FREQUENCY; FAILURES SCORE ZERO.

Regime	Designs	SFT	GRPO	Improved
Interpolation	14	69.8	214.8	14/14
Extrapolation	8	56.4	175.4	8/8
Overall	22	64.9	200.5	22/22

are empirical expected workloads, not measured runtime or speedup. They exclude policy-training cost and assume perfect post-route selection on observed support; they do not imply dominance over an unrestricted SFT search.

Source-feature aggregates give a complementary view across 16 designs with passing support in SFT, RF, and correctness-only. The mean maximum multiplications per statement falls from 8.776 for SFT to 1.377 for RF, while correctness-only reaches 17.335. RF also has higher structural accumulation and pipeline-ratio scores. These descriptive lexical differences are consistent with staged arithmetic and the measured FIR example. Because some features also enter the reward, they are not independent causal evidence or measurements of physical pipeline depth. The Supplementary Material reports the complete comparison and its weighting.

D. Backbone transfer and general RTL competence

The full training recipe also transfers to Qwen-Coder on the earlier 22-design, 3-family universe. Penalized F_{max} rises from 64.9 to 200.5 MHz, a paired gain of 135.6 MHz with interval [122.7, 147.3] MHz; all 22 designs improve. Correctness also rises from 80.1% to 93.0%. This supports transfer of the training recipe across backbones on the earlier family universe. Its reward configuration and split differ from the sealed RF experiment, so the two estimates remain separate.

The matched VerilogEval extension compares the actual sealed checkpoints on 156 official problems with 20 draws per checkpoint and problem. Pass-at-one is 22.34% for SFT,

TABLE VIII

MATCHED POST-PRIMARY GENERAL-RTL EVALUATION. EACH CHECKPOINT USES THE SAME BENCHMARK, GENERATION SETTINGS, AND DRAW BUDGET.

Checkpoint	Pass-at-one (%)
SFT	22.34
RF seed one	21.12
RF seed two	20.77

21.12% for RF seed one, and 20.77% for RF seed two (Table VIII). The RF-seed-mean paired change is -1.39 percentage points with interval [-2.69, -0.10], indicating a small additional regression on general RTL tasks. The interval resamples problems conditional on the evaluated checkpoints and does not estimate policy-training seed variance.

Within the accelerator domain, the RF checkpoint trajectory retains the physical gain from midpoint to endpoint. Interpolation changes from 125.5 to 125.4 MHz and extrapolation from 51.9 to 60.7 MHz. The Supplementary Material presents this trajectory alongside the earlier reward failure, keeping the evidence from the repaired run separate from the history that motivated it. Together with the matched VerilogEval result, these checks show why physical gains within the target domain and broader RTL competence should be evaluated separately. The Supplementary Material also provides a runnable FIR example and a completed RF versus SFT comparison on fresh draws with bounded routed setup-period measurements. These post-primary results retain their own measurement contract and are reported separately from the sealed primary estimate.

VI. DISCUSSION

A. Interpretation of the physical gain

The main result is that physical-reward training improves measured utility per draw on the sealed accelerator tasks. Both physical rewards improve over SFT, with the larger gain obtained by the RF package. This improvement comes from faster implementations among the outputs that pass the oracle. In contrast, correctness-only training increases acceptance while reducing physical utility. The additional-seed extension repeats both directions under its matched setup flow, and the separate Qwen-Coder experiment shows that the training recipe can also help another backbone on the earlier task set.

Changing candidate probabilities has practical value because it improves the expected hardware outcome at a fixed sampling budget. SFT already produces fast structures for many designs, and optimization makes a smaller set of passing outputs more likely. The structural summaries and implemented FIR pair support this interpretation, although they do not show that every gain arises from the same topology change. The finite samples also leave circuit novelty unresolved. Likewise, the perfect-selector calculation describes a separate inference workload and excludes policy-training cost, so it cannot establish whether reranking or policy training is the better overall strategy.

B. Implications for CAD workflows

A learned reward is useful when it can distinguish the physical quality of current policy outputs well enough to guide updates. The RF reward’s observed ranking resolution and sealed efficacy support that role in this domain, while the archived reward failure and predictor rescoring show why physical measurements remain necessary. The measured frequency gains also carry costs: RF uses more LUTs and registers, fewer DSP blocks, and slightly more vectorless power on common passing support. Pipeline latency can vary within the oracle’s alignment window, so a higher clock frequency does not necessarily reduce end-to-end latency. A deployment workflow would need explicit resource, latency, and power constraints, together with a check on general RTL competence given the matched VerilogEval regression.

The sampling budget and the total compute budget answer different questions. Matching draws measures the expected outcome of a fixed number of deployment attempts, but training arms can encounter different numbers of flat groups, each of which still requires generation and oracle execution. The perfect-selector comparison also excludes SFT, reward-label, and policy-training costs. Consequently, the present results do not identify the runtime, energy, or deployment volume at which policy training becomes more economical than repeated sampling and physical selection.

C. Scope and validity

The split evaluates unseen parameters within known streaming-accelerator generators. Its findings therefore do not establish competence on control-dominated systems, memories, multiple clocks, or free-form industrial RTL. Median filters appear only in extrapolation, which prevents us from separating their family effect from the regime effect. The Qwen study provides evidence on a second backbone but uses a narrower, earlier task universe. These scope limits also coexist with unknown overlap between the tested tasks and base-model pretraining data.

All RF and correctness-only training replications share one SFT initialization. Design-bootstrap intervals condition on the evaluated checkpoints and draws; they do not capture full-pipeline training variance. Family cells are small and unequal. RF versus SFT is the sole prespecified primary contrast; secondary intervals have no multiplicity adjustment. RF versus MLP changes predictor class, canonicalization, and features jointly. It supports a reward-package comparison, while individual component effects and matched external-method superiority remain untested. In particular, no matched DPO or recent external RTL-optimization baseline was executed.

The oracle supplies sampled, latency-aligned acceptance at a near-unity threshold. It does not prove exact functional equivalence. The original physical endpoint uses one device and a single requested-period flow, with constraints introduced after synthesis. Its slack variable has a zero default when a valid setup path is absent; the retained primary rows have not been reclassified under a stronger contract. The complete coverage diagnostic records one missing-path case and inconclusive implementations alongside close timing

agreement on common valid candidates. The period-bracketed pilot and additional-seed flow add stronger setup checks but do not enforce hold or pulse-width closure. The reverse-FIR pilot outlier also limits absolute-frequency calibration. Broader device, tool, and operating-corner transfer remains untested.

The RF repair hypothesis was motivated by earlier MLP outcomes. Its sealed efficacy split stayed unseen until checkpoints and the reward-eligible validation contract were frozen, but that contract followed an unsuccessful broader gate. The board subset likewise required feasibility and sweep amendments before measurement. These histories are retained in the Supplementary Material: the board result is descriptive, and the repair was prospectively tested after being motivated by earlier evidence. Contextual API and selected-candidate HLS comparisons have different reporting objects from the primary policy endpoint.

D. Artifact and claim traceability

The research archive retains the manifests, sampled RTL and multiplicities, oracle summaries, implementation failures, board traces, and numerical claim ledgers needed to trace the reported results.¹ The accompanying supplementary package provides the worked RTL example and recomputes the completed RF versus SFT comparison from its archived draws and physical records, with a file manifest linking the inputs to the analysis. Regeneration checks design coverage, draw denominators, implementation records, source hashes, and every numerical claim in the paper and supplement. Each completed supporting study retains its original evaluation contract. Prepared protocols, training-only seed logs, the asymmetric historical board comparison, and the incomplete smaller-student evaluation remain in the archive but do not enter the paper’s efficacy evidence.

VII. CONCLUSION

Training with a learned physical reward improves measured utility per RTL draw on the studied streaming-accelerator tasks under the sampled oracle contract. On the sealed split, the RF package increases failure-penalized post-route timing-derived frequency from 35.4 to 93.0 MHz, with positive effects for both policy-training seeds. The gain comes from faster oracle-passing implementations, despite a slightly lower pass rate. The original-MLP and single-seed correctness-only controls help explain this result without isolating the individual components of the RF repair. A separate setup-closure extension repeats the positive RF and negative correctness-only effects in two additional seeds per arm, all sharing the same SFT initialization. The Qwen-Coder study provides further evidence that the overall training recipe can benefit another backbone.

The benefits vary across families and come with implementation costs, including higher LUT and register use, slightly higher vectorless power, and a small regression in the matched VerilogEval experiment. The setup-closure extension and descriptive board measurements provide additional

¹Project repository: <https://github.com/MoAmineHallam/FPGA>.

physical evidence while retaining their separate contracts and limits; neither replaces the sealed endpoint or establishes full timing sign-off. Within these bounds, the results support using a learned physical reward to improve the sampling distribution, with executable testing, complete failure accounting, and independent physical measurements determining the reported outcome.

REFERENCES

- [1] M. Liu, N. Pinckney, B. Khailany, and H. Ren, “VerilogEval: Evaluating large language models for verilog code generation,” in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023.
- [2] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, “RTLML: An open-source benchmark for design RTL generation with large language model,” in *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024.
- [3] N. Pinckney, C. Batten, M. Liu, H. Ren, and B. Khailany, “Revisiting VerilogEval: A year of improvements in large-language models for hardware code generation,” *ACM Transactions on Design Automation of Electronic Systems*, vol. 30, no. 6, 2025.
- [4] Y. Zhao, W. Fu, S. Li, Y.-X. Hu, X. Guo, and Y. Jin, “Hardware generation with high flexibility using reinforcement learning enhanced LLMs,” in *Proceedings of the 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025.
- [5] Z. Chen, K. Chang, Z. Li, C. Li, X. He, C. Chen, M. Wang, H. Xu, Y. Han, H. Li, and Y. Wang, “ChipSeek: Optimizing verilog generation via EDA-integrated reinforcement learning,” in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 25 180–25 201.
- [6] X. Yao, Y. Li, T. H. Zhang *et al.*, “RTLRewriter: Methodologies for large models aided RTL code optimization,” in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024.
- [7] H. Ping, P. Zhang, Z. Wang, S. Li, A. Cheng, W. Yang, P. Bogdan, and S. Nazarian, “POET: Power-oriented evolutionary tuning for LLM-based RTL PPA optimization,” *arXiv preprint arXiv:2603.19333*, 2026.
- [8] W.-P. Hsin, R.-H. Deng, Y.-T. Hsieh, E.-M. Huang, and S.-H. Hung, “EvoVLE: Evolutionary search for LLM-based verilog generation and optimization,” *arXiv preprint arXiv:2601.18067*, 2026.
- [9] P. Zhou, Z. Chen, C. Li, H. Gao, K. Chang, Z. Qu, and Y. Wang, “Alpha-RTL: Test-time training for RTL hardware optimization,” *arXiv preprint arXiv:2606.05253*, 2026.
- [10] W. Fang, Y. Lu, S. Liu, J. Wang, Z. Guo, J. He, F. Tu, and Z. Xie, “Dr. RTL: Autonomous agentic RTL optimization through tool-grounded self-improvement,” *arXiv preprint arXiv:2604.14989*, 2026.
- [11] W. Fu, Z. Wang, M. Shao, R. Karri, M. Shafique, J. Knechtel, O. Sinanoglu, and X. Guo, “Synthesis-in-the-loop evaluation of LLMs for RTL generation: Quality, reliability, and failure modes,” *arXiv preprint arXiv:2603.11287*, 2026.
- [12] L. Gao, J. Schulman, and J. Hilton, “Scaling laws for reward model overoptimization,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023, arXiv:2210.10760.
- [13] B. Hui, J. Yang, Z. Cui *et al.*, “Qwen2.5-Coder technical report,” *arXiv preprint arXiv:2409.12186*, 2024.
- [14] Y. Tan, J. Du, X. Meng, and Y. Lyu, “ChipVerilog: A large-scale OpenCores-derived benchmark for LLM-based verilog RTL generation,” *arXiv preprint arXiv:2607.13079*, 2026.
- [15] S. Liu, Y. Lu, W. Fang, M. Li, and Z. Xie, “OpenLLM-RTL: Open dataset and benchmark for LLM-aided design RTL generation: Invited paper,” in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024, pp. 1–9.
- [16] D. Garcia-Gasulla, G. Kestor, E. Parisi, M. Alberti-Binimelis, C. Gutierrez, R. M. Ghorab, O. Montenegro, B. Homs, and M. Moreto, “TuRTL: A unified evaluation of LLMs for RTL generation,” in *ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD)*, 2025, arXiv:2504.01986.
- [17] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, “VeriGen: A large language model for verilog code generation,” *ACM Transactions on Design Automation of Electronic Systems*, 2024.
- [18] S. Liu, W. Fang, Y. Lu, Q. Zhang, H. Zhang, and Z. Xie, “RTLCoder: Outperforming GPT-3.5 in design RTL generation with our open-source dataset and lightweight solution,” in *IEEE International Workshop on LLM-Aided Design (LAD)*, 2024.
- [19] Y. Zhao, D. Huang, C. Li, P. Jin *et al.*, “CodeV: Empowering LLMs for verilog generation through multi-level summarization,” *arXiv preprint arXiv:2407.10424*, 2024.
- [20] M. Liu, Y.-D. Tsai, W. Zhou, and H. Ren, “CraftRTL: High-quality synthetic data generation for verilog code models with correct-by-construction non-textual representations and targeted code repair,” *arXiv preprint arXiv:2409.12993*, 2024.
- [21] N. Wang, B. Yao, J. Zhou, X. Wang, Z. Jiang, and N. Guan, “Large language model for verilog generation with code-structure-guided reinforcement learning,” *arXiv preprint arXiv:2407.18271*, 2024.
- [22] Y. Wang, G. Sun, W. Ye, G. Qu, and A. Li, “VeriReason: Reinforcement learning with testbench feedback for reasoning-enhanced verilog generation,” *arXiv preprint arXiv:2505.11849*, 2025.
- [23] N. Wang, B. Yao, J. Zhou, Y. Hu, X. Wang, Z. Jiang, and N. Guan, “Insights from verification: Training a verilog generation LLM using reinforcement learning with testbench feedback,” in *ACM/IEEE Design Automation Conference (DAC)*, 2026, preprint: arXiv:2504.15804.
- [24] K. Li, M. Li, X. Wen, S. Zhao, J. Wu, J. Huang, and Q. Xu, “FormalRTL: Verified RTL synthesis at scale,” *arXiv preprint arXiv:2603.08738*, 2026.
- [25] X. Kong, T. Tabassum, M. Abdelwahab, and H. Zheng, “A progressive approach to synthesizable RTL design generation using LLMs,” *arXiv preprint arXiv:2607.18519*, 2026.
- [26] Z. Pei, H. Zhen, M. Yuan, Y. Huang, and B. Yu, “BetterV: Controlled verilog generation with discriminative guidance,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 40 145–40 153.
- [27] Y. Wang, W. Ye, P. Guo, Y. He, Z. Wang, B. Tian, S. He, G. Sun, Z. Shen, S. Chen, A. Srivastava, Q. Zhang, G. Qu, and A. Li, “Sym-RTL: Enhancing RTL code optimization with LLMs and neuron-inspired symbolic reasoning,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025.
- [28] K. Min, K. Cho, J. Jang, and S. Kang, “REvolution: An evolutionary framework for RTL generation driven by large language models,” in *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2026, arXiv:2510.21407.
- [29] H. Ping, P. Zhang, S. Li, W. Yang, A. Cheng, S. Duan, X. Zhang, and P. Bogdan, “COEVO: Co-evolutionary framework for joint functional correctness and PPA optimization in LLM-based RTL generation,” *arXiv preprint arXiv:2604.15001*, 2026.
- [30] Y. Lu, S. Liu, H. Zhou, W. Fang, Q. Zhang, and Z. Xie, “A new benchmark for the appropriate evaluation of RTL code optimization,” *arXiv preprint arXiv:2601.01765*, 2026.
- [31] E. Ustun, C. Deng, D. Pal, Z. Li, and Z. Zhang, “Accurate operation delay prediction for FPGA HLS using graph neural networks,” in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2020.
- [32] N. Wu, H. Yang, Y. Xie, P. Li, and C. Hao, “High-level synthesis performance prediction using GNNs: Benchmarks, modeling, and advancing,” in *ACM/IEEE Design Automation Conference (DAC)*, 2022.
- [33] W. Fang, Y. Lu, S. Liu, Q. Zhang, C. Xu, L. W. Wills, H. Zhang, and Z. Xie, “Transferable pre-synthesis PPA estimation for RTL designs with data augmentation techniques,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 2024.
- [34] E. J. Hu, Y. Shen, P. Wallis *et al.*, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [35] Z. Shao, P. Wang, Q. Zhu *et al.*, “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models,” *arXiv preprint arXiv:2402.03300*, 2024, introduces Group Relative Policy Optimization (GRPO).